

MSPM: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management

Zhenhan Huang, Fumihide Tanaka

Published: February 18, 2022 • <https://doi.org/10.1371/journal.pone.0263689>

Correction

17 Mar 2022: Huang Z, Tanaka F (2022) Correction: MSPM: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management. PLOS ONE 17(3): e0265924. <https://doi.org/10.1371/journal.pone.0265924> | [View correction](#)

Abstract

Financial portfolio management (PM) is one of the most applicable problems in reinforcement learning (RL) owing to its sequential decision-making nature. However, existing RL-based approaches rarely focus on scalability or reusability to adapt to the ever-changing markets. These approaches are rigid and unscalable to accommodate the varying number of assets of portfolios and increasing need for heterogeneous data input. Also, RL agents in the existing systems are ad-hoc trained and hardly reusable for different portfolios. To confront the above problems, a modular design is desired for the systems to be compatible with reusable asset-dedicated agents. In this paper, we propose a multi-agent RL-based system for PM (MSPM). MSPM involves two types of asynchronously-updated modules: Evolving Agent Module (EAM) and Strategic Agent Module (SAM). An EAM is an information-generating module with a Deep Q-network (DQN) agent, and it receives heterogeneous data and generates signal-comprised information for a particular asset. An SAM is a decision-making module with a Proximal Policy Optimization (PPO) agent for portfolio optimization, and it connects to multiple EAMs to reallocate the corresponding assets in a financial portfolio. Once been trained, EAMs can be connected to any SAM at will, like assembling LEGO blocks. With its modularized architecture, the multi-step condensation of volatile market information, and the reusable design of EAM, MSPM simultaneously addresses the two challenges in RL-based PM: scalability and reusability. Experiments on 8-year U.S. stock market data prove the effectiveness of MSPM in profit accumulation by its outperformance over five different baselines in terms of accumulated rate of return (ARR), daily rate of return (DRR), and Sortino ratio (SR). MSPM improves ARR by at least 186.5% compared to constant rebalanced portfolio (CRP), a widely-used PM strategy. To validate the indispensability of EAM, we back-test and compare MSPMs on four different portfolios. EAM-enabled MSPMs improve ARR by at least 1341.8% compared to EAM-disabled MSPMs.

Citation: Huang Z, Tanaka F (2022) MSPM: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management. PLoS ONE 17(2): e0263689. <https://doi.org/10.1371/journal.pone.0263689>

Editor: Thippa Reddy Gadekallu, Vellore Institute of Technology: VIT University, INDIA

Received: November 10, 2021; **Accepted:** January 24, 2022; **Published:** February 18, 2022

Copyright: © 2022 Huang, Tanaka. This is an open access article distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Data Availability: The stock prices data for this study were from Nasdaq Data Link provided by QuoteMedia, which can be accessed by subscribing to: <https://data.nasdaq.com/data/EOD-end-of-day-us-stock-prices>. The authors subscribed to those data, and do not have any special access privileges. The stock sentiments data for this study were obtained from Nasdaq Data Link provided by InfoTrie, which can be accessed by subscribing to: <https://data.nasdaq.com/databases/NS1/data>.

Funding: The author(s) received no specific funding for this work.

Competing interests: The authors have declared that no competing interests exist.

Introduction

Portfolio management (PM) is a continuous process of reallocating capital into multiple assets [1], and it aims to maximize accumulated profits with an option to minimize the overall risks of the portfolio. To perform such a practice, portfolio managers who focus on stock markets conventionally read financial statements and balance sheets, follow the news from media and announcements from financial institutions and analyze stock price trends. By the resemblant nature of the problem, researchers expectedly wish to incorporate deep reinforcement learning (DRL) methods in PM. As one of the attempts, the authors of [2] propose a PM framework for cryptocurrencies using Deep Deterministic Policy Gradient (DDPG) [3, 4]. [5] proposes a method called Adversarial Training for portfolio optimization with the implementation of three different RL methods: DDPG, Proximal Policy Optimization (PPO) [6] and Policy Gradient (PG). Akin to receiving information from various sources as portfolio managers generally do, existing approaches incorporate heterogeneous data [7]. Recently, multi-agent reinforcement learning (MARL) approaches are also proposed by researchers [8–10]. In [10], the authors propose MAPS, a system involving a group of Deep Q-network [11] (DQN)-based agents corresponding to individual investors, to make investment decisions and create a diversified portfolio. MAPS can be recognized as a reinforcement-learning implementation of ensemble learning [12] by its very nature. In addition, [13] proposes iRDPG to generate adaptive quantitative trading strategies by using DRL and imitation learning. However, while inspiring, the existing approaches seldom focus on scalability and reusability to accommodate the ever-changing markets. RL

agents in the existing multi-agent-based systems are ad-hoc trained and rarely reusable for different portfolios. Also, the existing systems are barely scalable to answer the need for scaled number of assets in portfolios and increasing heterogeneous data input. For example, in SARL [7], the encoder's intake is either financial news data for embedding or stock prices for trading signals generation, but can not be both of them, and this issue prevents the encoder from efficiently producing holistic information and eventually limits the RL-based agents' learning. Furthermore, the existing systems lack a modular design to be compatible with different RL agents for different assets. In this paper, we propose MSPM, a novel multi-agent reinforcement learning-based system, with a modularized and scalable architecture for PM. In MSPM, assets are vital and organic building blocks. This vitalness is reflected in that each asset has its dedicated module: Evolving Agent Module (EAM). An EAM takes heterogeneous data and utilizes a DQN-based agent to produce signal-comprised information. After we set up and trained the EAMs corresponding to the assets in a portfolio, we connected them to a decision-making module: Strategic Agent Module (SAM). An SAM represents a portfolio and uses the profound information from the connected EAMs for asset reallocation. EAM and SAM are asynchronously updated, and EAMs' reusability allows themselves to be combined and connected to multiple SAMs discretionarily. With the power of parallel computing, we can perform capital reallocation for various portfolios at scale, simultaneously.

To evaluate MSPM's performance, we back-test and compare MSPM to five different baselines on two different portfolios. MSPM outperforms all the baselines in terms of accumulated rate of return, daily rate of return, and Sortino ratio. For instance, MSPM improves accumulated rate of return by 49.3% and 426.6% compared to the state-of-the-art RL-based method: Adversarial PG [5] on the two portfolios. We also inspect the position-holding of five different EAMs to exemplify the high quality and reliability of the signals generated by EAM. Specifically, the average winning rate of the EAMs in the two portfolios achieves 80%. Furthermore, we validate the necessity of EAM by back-testing and comparing the EAM-enabled and disabled MSPMs on four different portfolios. EAM-enabled MSPMs improve accumulated rate of return by at least 1341.8% compared to the EAM-disabled MSPMs. Our contribution can be listed as follows:

- To the best of our knowledge, MSPM is the first approach that formalizes a modularized and scalable multi-agent reinforcement learning system using signal-comprised information for financial portfolio management.
- MSPM with its modularized and reusable design addresses the issue of ad-hoc, fixed, and inefficient model training in the existing RL-based methods.
- By experiment and comparison, we confirm that our MSPM system outperforms five different baselines under extreme market conditions of U.S. stock markets during the global pandemic, from January to December 2020.
- EAM-enabled MSPM systems improve accumulated rate of return of two different portfolios by 49.3% and 426.6% compared to Adversarial PG [5], a state-of-the-art RL-based method, and by 186.5% and 369.8% compared to Constant Rebalanced Portfolio (CRP)[14], a conventional PM strategy. In addition, the average winning rate of the EAMs in the two portfolios achieves 80%.
- Furthermore, we validate the indispensability of Evolving Agent Module (EAM) by back-testing MSPM on four different investment portfolios. Among the portfolios, EAM-enabled MSPMs improve accumulated rate of return by at least 1341.8% compared to the EAM-disabled MSPMs.

Related work

In the early years, researchers and professionals believe that certain behaviors of price and volume will repeat periodically and consistently. Based on this recognition, the technical indicators (TI) are invented by using historical price and volume data to predict the movement of asset prices [15]. TIs are mostly formulas or particular patterns, and the trading strategies that utilize TIs are referred to as technical analysis (TA)[16]. However, as pre-defined formulas and patterns cannot cover all market movements, it is getting harder and harder for TA to adapt to the fast-changing market. With the increase in computing power and available data, researchers have started to use deep learning (DL) to predict stock price movements. DL uses high-dimensional data to train complex and non-linear neural network models as trading strategies. Fortunately, DL's adaptability to the market is promisingly improved compared to TA. Recently, deep reinforcement learning (DRL) has emerged rapidly as the combination of DL and reinforcement learning (RL). By utilizing neural networks (NN), a DRL-based agent is particularly good at extracting useful information from high-dimensional data and taking sequential actions based on rewarding. DRL methods have led to many breakthroughs in multiple fields. For instance, [11] successfully utilizes Deep Q-learning agents to learn directly from high-dimensional raw pixel input to play video games. Due to the sequential decision-making nature of financial investment, researchers naturally attempt to solve stock trading problems using DRL methods. [2] designed a cryptocurrencies portfolio management (PM) framework using Deep Deterministic Policy Gradient (DDPG)[3, 4] which is a model-free DRL algorithm. [5] proposes the Adversarial Training method to improve training efficiency using three different RL methods: DDPG, Proximal Policy Optimization (PPO)[6] and Policy Gradient (PG). Although these approaches have presented potential performance, the data input of these approaches is still traditional historical data, namely opening-high-low-closing prices (OHLC) and trading volumes. Unlike preceding research, [7] proposes SARL, an RL framework that can incorporate heterogeneous data to generate PM strategies. Moreover, to address the challenge of balancing between exploration and exploitation, [13] proposes iRDPG for developing trading strategies by DRL and imitation learning. Multi-agent systems have also been proposed. In [10], the authors propose MAPS, a cooperative system containing multiple agents, to create diversified portfolios and to adapt to the continuously changing market conditions. However, while the existing approaches tackle PM problems with promising methods and techniques, these systems, with the strategies generated, are mostly fixed and ad-hoc. The existing systems or frameworks lack a modular design to be compatible with different trained RL agents. The RL agents trained for one portfolio can hardly be reused for different portfolios. These systems also lack scalability to accommodate the increasing number of assets and profundity of market information. In this paper, we propose MSPM for solving the problems.

Data

Data acquisition

The historical price data used in this paper are QuoteMedia's End of Day US Stock Prices (EOD) [17] from Jan 2013 to Dec 2020 obtained using Nasdaq Data Link's API, which can be accessed by subscribing at: <https://data.nasdaq.com/data/EOD-end-of-day-us-stock-prices>. We also use web news sentiment data (FinSentS)[18] from Nasdaq Data Link provided by InfoTrie, which can be accessed by subscribing at: <https://data.nasdaq.com/databases/NS1/data>.

Feature selection and data curation

We select the adjusted- close, open, high, and low prices and volumes features from QuoteMedia's EOD data as the historical price data. We also select the sentiment and news_buzz from InfoTrie's FinSentS Web News Sentiment. Each feature in EOD data is normalized by dividing the first (day-one) value of that feature, and there is no missing value in any of these features. For FinSentS

data, we use original values of the sentiment feature in FinSentS data, and we fill the missing values (accounting for 9.51% of the total data) prior year 2013 with a neutral sentiment: zero (0). Since the FinSentS data are not as straightforward as EOD data, we put the description of the selected features of FinSentS data in [Table 1](#).

Feature	Description
sentiment	A measure of bullishness and bearishness of equity prices calculated as a statistical index of the news corpus. Sentiment scores are defined on a scale of -5 to 5 indicating from the most bearish to the most bullish.
news_buzz	Normalized value of change in standard deviations of periodic number of news items (news volume) used for generating sentiment. Buzz scores reflect a sharp change in news volume thus serving as a risk alert indicator. Defined on a scale of 1–10 high buzz score reflects higher volatility.

<https://doi.org/10.1371/journal.pone.0263689.t001>

Table 1. Feature selection of FinSentS data.

<https://doi.org/10.1371/journal.pone.0263689.t001>

Methodology

Our MSPM system consists of two types of modules: EAM and SAM. The relationship between EAMs and SAMs is illustrated in [Fig 1](#). [Fig 2](#) illustrates a even more intuitive overview of MSPM's architecture. To accommodate MSPM in the sequential decision-making problems financial portfolio management, we configured the specific settings for EAM and SAM. An EAM contains a DQN **agent** and **acts** to generate signal-comprised information (historical prices with buy/closing/skip labels) for a designated asset. To train the agent in EAM, we constructed a sequential decision-making problem with designated asset's historical prices and financial news as the **state** that the agent observes at each time step. An DQN agent acts to buy or close a position, or simply to skip at every time step based on the latest prices and financial news data input, in order to maximize its total reward. The actions (signals) then will be matched and stacked back to the corresponding price data to formalize the signal-comprised information. EAM's architecture is illustrated in [Fig 3](#). On the other hand, an SAM manages an investment portfolio and contains a PPO **agent** that reallocates the assets in that portfolio. SAMs are connected to multiple EAMs as an investment portfolio often has more than one asset. In the decision-making process of SAM, the state that the PPO agent **observes** at each time step is the combination of the signal-comprised information which the connected EAMs generate. Further, the PPO agent **acts** to generate the reallocation weights for the assets in the portfolio, which total up to 1.0. [Fig 4](#) provides an overview of the SAM's architecture. For both EAM and SAM, the composition of the assets' historical prices and financial news or news sentiments is the **environment** their agents interact with. Each EAM is reusable. Once an EAM is set up and trained, it can be effortlessly connected to any SAM. An SAM connects to at least one EAM. EAMs are retrained periodically using the latest information from the market, media, financial institutions, etc., and we implemented the former two data sources in this study. In the following sections, we explain the technical details of EAM and SAM.

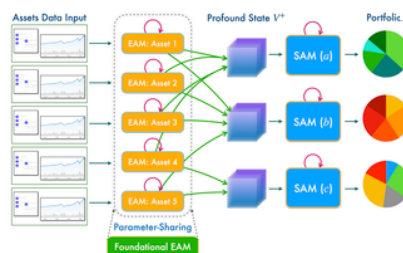


Fig 1. Overview of the surjection relationship between Evolving Agent Modules (EAMs) and Strategic Agent Modules (SAMs).

Each EAM is responsible for a single asset and employs a DQN agent, and it utilizes heterogeneous data to produce signal-comprised information. Each SAM is a module for a portfolio that employs a PPO agent to reallocate the assets with stacked signal-comprised 3-D tensor **profound state** V^* from EAMs connected. Moreover, trained EAMs are reusable for different portfolios and therefore can be combined and connected to any SAMs at will. By parallel computing, capital reallocation may be performed for various portfolios at scale simultaneously.

<https://doi.org/10.1371/journal.pone.0263689.g001>

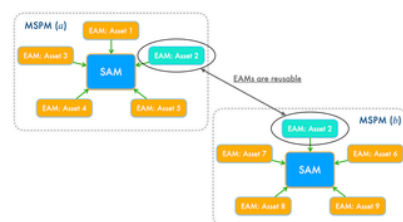


Fig 2. A more intuitive illustration of MSPM's architecture.

EAMs are reusable for different portfolios. EAMs can be combined and connected to any SAMs at will, like assembling LEGO blocks.

<https://doi.org/10.1371/journal.pone.0263689.g002>

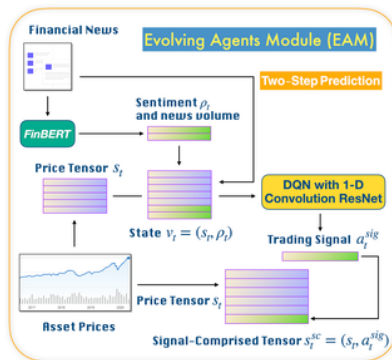


Fig 3. Abstract of EAM's architecture.

An EAM is a module for a designated asset. Each EAM takes two types of heterogeneous data: 1. designated asset's historical prices and 2. asset-related financial news. At the center of an EAM is an extended DQN agent using a 1-D convolution ResNet for sequential decision making. Instead of training every EAM from scratch, we train EAMs by transfer learning using a foundational EAM. At every time step t , the DQN agent in EAM observes state v_t of historical prices s_t and news sentiments ρ_t of the designated asset, and acts to trade with an action of either buying, selling, or skipping, and eventually generates a 2-D signal-comprised tensor using new prices s_t and signals.

<https://doi.org/10.1371/journal.pone.0263689.g003>

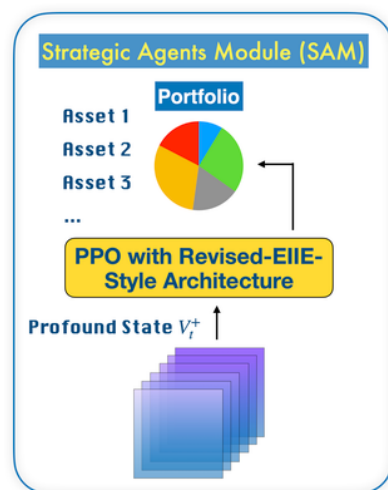


Fig 4. Abstract of SAM's architecture.

An SAM is a module for an investment portfolio. The input of SAM, profound state, is a 3-D tensor, where f is the number of features, $m^* = m + 1$ is the number of assets m in the portfolio plus cash and n is the fixed rolling-window length. Each SAM takes the profound state which is stacked and transformed from 2-D tensors from connected EAMs, and further generates the reallocation weights for the assets in the portfolio.

<https://doi.org/10.1371/journal.pone.0263689.g004>

Evolving Agent Module (EAM)

State.

At any given periodic (daily) time-step t , the agent in EAM observes state v_t , which consists of the designated asset's recent n -day historical prices s_t and sentiment scores ρ_t . Specifically,

(1)

where s includes the designated asset's n -day close, open, high and low prices and volumes. ρ includes the predicted and averaged news sentiments, using a pre-trained FinBERT classifier [19, 20] for asset-related financial news, which ranges continuously from -5.0 to 5.0, indicating bearishness (-5.0) or bullishness (5.0). Furthermore, ρ also includes *news_buzz*. This attribute is an attempt to alleviate the unbalanced-news issue in the existing research [7]. Instead of restarting from the beginning after every episodic reset of the environment, the environment resets at a random time point of the data [21].

Because the news sentiments from FinSentS data and the sentiments generated by FinBERT are similar, and due to the restriction of APIs and web scraping, we only utilize FinSentS data as the sentiments input for the experiments in this paper.

Deep Q-network.

For an EAM, we train a Deep Q-network (DQN) agent and follow the sequential decision-making of Deep Q-learning [11]. Deep Q-learning is a value-based method that derives a deterministic policy $\pi(\theta)$, which is a mapping: $S \rightarrow A$ from state space to discrete action space. We use a Residual Network with 1-D convolution [22] to represent the state-value function Q^θ which the agent acts based on:

(2)

For information about model selection for EAM and hyperparameter tuning, see [S1 Appendix](#).

DQN extensions. We implement three extensions [21] of the original DQN, namely dueling architecture [23], Double DQN [24] and two-step Bellman unrolling.

Transfer learning. Instead of training every EAM from scratch, we initiate and train a **foundational EAM**, using historical prices of AAPL (Apple Inc.), and then train all other EAMs based on this pre-trained EAM. By doing so, the foundational EAM shares its parameters with other EAMs which obtains prior knowledge of the pattern of stock trends. This transfer learning approach may help to tackle the **data-shortage** issue of newly-listed stocks due to the limited historical prices and news data available for training purposes.

Action.

The DQN agent in EAM acts to trade the designated asset with an action of either buying, selling, or skipping, at every time step t . The choice of an action, $a_t = \{\text{buying, closing, or skipping}\}$, is called an **asset trading signal**. As indicated in the actions, there is no short (selling) position, and a new position will be opened only after an existing position has been closed.

Reward.

The reward, r_t , received by the DQN agent at each time step t is:

(3)

where p_t is the close price of the given asset at time step t . t_l is the time step when a long position is opened and commissions are deducted, β stands for the commission of 0.0025 and I_t is the indicator of an opening position (i.e., a position is still open).

Strategic Agent Module (SAM)

State (stacked signal-comprised tensor).

Once EAMs have been trained, we feed new historical prices, s_t , and financial news of the designated assets, to generate predictive trading signals. Then we stack the same new historical prices to formalize a 2-D signal-comprised tensor as the data source to train SAM. Because an SAM is connected to multiple EAMs, the 2-D signal-comprised tensors from all connected EAMs are stacked and transformed into a 3-D signal-comprised tensor called **profound state**, which is the state that SAM observes at each time step t .

Proximal policy optimization.

A PPO [6] agent is at the center of SAM to reallocate assets. PPO is an actor-critic style policy gradient method that has been widely used on continuous action space problems, due to its desirable performance and ease of implementation. A policy π_θ is a parametrized mapping: $S \times A \rightarrow [0, 1]$ from state space to action space. Among the different objective functions of PPO, we implement the clipped surrogate objective [6]:

(4)

where

and A_t , the advantage function, is expressed as:

in which, the state-action value function Q^θ is:

and the value function V^θ is:

For the PPO agent, we design a policy network architecture targeting the uniqueness of continuous action space in financial portfolio management problems, inspired by the EIE topology [2]. Because assets' reallocated weights at time step t are strictly required to total up to 1.0, we set m^* normal distributions, and we sample from the distributions, where $m^* = m + 1$ and z is the linear output of the last layer of the neural network and with standard deviation $\sigma = 0$. We eventually obtain the reallocation weights $a_t = \text{Softmax}(x_t)$ and the log probability of x_t for the PPO agent to learn.

Fig 5 shows the details of the policy network (actor) of SAM, denoted by θ' . Due to the resemblance and equivalence, architectures of the value network (critic) and target policy network, denoted by θ , are not illustrated.

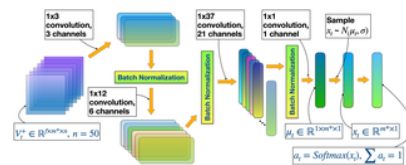


Fig 5. Policy network (θ) of SAM to accommodate PPO algorithm.

Profound state s_t is the input of the network. f is the number of features, m^* is the number of assets in the portfolio, and $n = 50$ is the fixed rolling-window length. After s_t are sampled from the normal distributions, we calculate log probability of x_t and obtained the reallocation weights $a_t = \text{Softmax}(x_t)$. ReLu activation function [25] is set after every convolutional layer, except the last one.

<https://doi.org/10.1371/journal.pone.0263689.g005>

Action.

The action the PPO agent takes at each time step t is

(5)

which is the vector of reallocating weights at each time step t , and Fig 6 shows the details of price fluctuations.

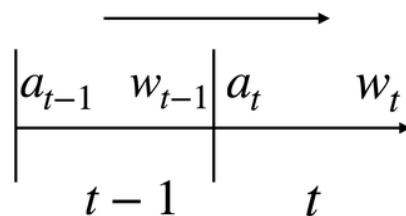


Fig 6. Transformed allocation weights due to the fluctuation in assets' prices.

<https://doi.org/10.1371/journal.pone.0263689.g006>

Once the assets are reallocated by a_t , the allocation weights of the portfolio eventually become

(6)

at the end of time step t due to the price fluctuation during the time step period; where,

(7)

is the relative price vector, that is, the changes of asset prices over time, including the prices of assets and cash. $p_{i,t}$ denotes the closing price of the i -th asset at time t , where $i = \{2, \dots, m^*\}$, excluding cash (risk-free asset) whose closing price should always be 1.

Reward.

Inspired by [2] in which the agent maximizes the sum of the logarithmic value, and [5] in which the authors try to cluster the periodic portfolio risk to alleviate the biases in training data and to prevent exposure to highly-volatile assets, we set the reward to be a risk-adjusted rate of return, r_t , which PPO agent receives at each time step t .

(8)

where m^* is the number of assets, w_t represents the allocation weights of the assets at the end of time step t .

(9)

is the transaction cost, where $\beta = 0.0025$ is the commission rate, and $\varphi = 0.001$ is the risk discount which can be fine-tuned as a hyperparameter.

(10)

measuring the volatility of fluctuation in assets' prices during the last n days.

(11)

is the volatility of the profit of an individual asset. We expect the agent to secure a maximum risk-adjusted rate of return (capital gain) every time step, as what is expected from human portfolio managers.

Experiments

In this section, we build different portfolios, and train MSPM to periodically reallocate the assets in each portfolio. The portfolios, datasets, and performance metrics for benchmarking will be introduced and described. After that, we explain and discuss the experimental results and examine MSPM's stability of daily rate of return. We also inspect the signal generation and position-holding of EAMs. In the end, we validate the necessity of EAM by back-testing four different portfolios. The back-testing performance of MSPM will be compared with the existing baselines.

Preliminaries

Portfolios.

We first propose two portfolios: (a) and (b) to compare back-testing performance. Portfolio(a) includes three stocks: Apple, AMD, and Alphabet (symbol codes: [AAPL, AMD, GOOGL]), and Portfolio(b) includes three other stocks: Alphabet, Nvidia, and Tesla (symbol codes: [GOOGL, NVDA, TSLA]). To build portfolio(a) and portfolio(b), we trained two SAM/MSPMs: SAM/MSPM(a) and SAM/MSPM(b). Additionally, the two SAMs shared the same EAM for the stock in common: Alphabet (GOOGL). Later, we propose two other portfolios (c) and (d), which make four portfolios in total, to validate the necessity of EAM. Details can be found in the Validation of EAM section. For all these four portfolios, we set initial portfolio value to be $p_0 = 10,000$.

Data ranges.

Among the EAMs to be trained, the foundational EAM (AAPL) is trained initially, and its parameters are shared with other EAMs as their foundation for transfer learning. As shown in Table 2, EAM-training data, ranging from January 2009 to December 2015, contains the historical prices (s_t) and news sentiments (ρ_t) of the stocks, including AAPL, in portfolios (a) and (b). EAM-predicting data, with the same data structure as EAM-training and ranging from January 2016 to December 2020, is used for EAMs to predict and generate trading signals (actions of DQN agents). Then, EAM-predicting data along with the generated trading signals became the signal-comprised data for SAM/MSPMs. There are three datasets of signal-comprised data: SAM/MSPM-training and SAM/MSPM-validating to train and validate SAMs, respectively; and SAM/MSPM-experiment, from January 2020 to December 2020, for back-testing and other experiments. Details can be found in Table 2. It is worth noting that a low percentage (9.51%) of missing values from the alternative data (sentiments) shall not affect MSPM's scalability nor reusability since, as a general framework, MSPM is neutral on the structures, types, or sources of the data input.

Table 2. Date ranges of the data.

<https://doi.org/10.1371/journal.pone.0263689.t002>

Performance metrics.

We use the following performance metrics to measure the performances of the baselines and MSPM system.

► Daily Rate of Return (DRR)

(12)

where T is the terminal time step, and

(13)

is the risk-unadjusted periodic (daily) rate of return obtained at every time step, where c is the transaction cost and $\beta = 0.0025$ is the commission rate.

► Accumulated rate of return (ARR) The accumulated rate of return (ARR) [26] is

(14)

where T is the terminal time step, p_0 is the portfolio value at the initial time step, and

(15)

which stands for the portfolio value at the terminal time step.

► Sortino ratio (SR) Sortino ratio [27] is often referred to as a risk-adjusted return, which measures the portfolio performance compared to a risk-free return, adjusted by the portfolio's downside risk. In our case, Sortino ratio is calculated as

(16)

where R_t is the risk-unadjusted periodic (daily) rate of return. Portfolio's downside risk $\sigma^{downside}$ is calculated as

(17)

where R_f is the risk-free return and conventionally equals zero, R_t are the less-than-zero returns in R_t for all t , and $t = T$ is the terminal time step.

- **Max drawdown (MD)** MD is the biggest drop (in %) between the highest (peak) and lowest (valley) of the accumulated rate of return of a certain period of time.

For DRR, ARR and SR, we want them to be as high as possible, whereas we want MD to be as low as possible.

Results and discussion

Back-testing performance

We back-test and compare the performance of our MSPM system to different baselines, including the traditional and cuttings-edge RL-based portfolio management strategies [28, 29]. The baselines are listed as follows:

- **CRP** stands for (Uniform) Constant Rebalanced Portfolio, which involves investing an equal proportion of capital in each asset, namely $1/N$, which seems simple but, in fact, challenging to beat [14].
- **Buy and hold (BAH)** strategy involves investing without rebalancing. Once the capital is invested, no further allocation will be made.
- **Exponential gradient portfolio (EG)** strategy involves investing capital into the latest stock with the best performance and uses a regularization term to maintain the portfolio information.
- **Follow the regularized leader (FTRL)** strategy tracks the Best Constant Rebalanced Portfolio until the previous period, with an additional regularization term. This strategy reweights based on the entire history of the data with an expectation to obtain maximum returns.
- **ARL** refers to the adversarial deep reinforcement learning in portfolio management (Adversarial PG) [5], which is a state-of-the-art (SOTA) RL-based portfolio management method.

As shown in Figs 7 and 8, for both portfolios (a) and (b), MSPM system improves ARR, by at least 49.3% and 426.6% compared to ARL, a SOTA RL-based PM method, and by 186.5% and 369.8% compared to CRP, a traditional PM strategy, during the year of 2020. The result demonstrates the advantage of MSPM at gaining capital returns. Table 3 gives details about MSPM's outperformance over existing baselines in terms of the ARR and DRR. Further, MSPM's superior performance on SR indicates that MSPM takes better consideration of harmful volatility and achieves higher risk-adjusted returns.

Fig 7. MSPM(a) outperforms all baselines on Portfolio(a) in terms of the accumulated portfolio value in back-testing.
<https://doi.org/10.1371/journal.pone.0263689.g007>

Fig 8. MSPM(b) outperforms all baselines on Portfolio(b) in terms of the accumulated portfolio value in back-testing.
<https://doi.org/10.1371/journal.pone.0263689.g008>

Table 3. Comparison of back-testing performance of the baselines and MSPM.
<https://doi.org/10.1371/journal.pone.0263689.t003>

It is worth noting that for portfolio (a), both MSPM and ARL achieve promising SR, but for portfolio (b), only MSPM has a much better Sortino ratio than ARL, which indicate MSPM's higher adaptability to the ever-changing market compared to not only the traditional strategies but also the preceding RL-based method.

Stability of daily rate of return (DRR)

Due to the high max drawdown (MD) of MSPM for portfolio(b) (60.6%), we want to examine and compare the general stability of DRR between MSPM and the state-of-the-art RL-based method: ARL. For this purpose, we first calculate DRR's 5-day rolling standard deviation (RstdDRR) as the proxy of the stability of DRR. Higher RstdDRR indicates lower stability of DRR.

To calculate the RstdDRR, we first calculate the simple moving average (SMA) [30] of for the past n data-points (days) by the following formula:

(18)

for $i = n, \dots, k$. Then, we subtract SAM_i from the 5-day DRRs used in the calculation, and then take the square root of the squared summation to have the rolling standard deviation: :

(19)

where $i = n, \dots, k$.

Fig 9 shows the histograms of MSPM and ARL's RstdDRR for portfolio(a), and histograms in Fig 10 are for portfolio(b). According to Fig 9, the right tail of ARL's RstdDRR is fatter than that of MSPM's RstdDRR, and MSPM has a lower average RstdDRR ($M_{(a)} = 0.031$, $SD_a = 0.019$) than ARL ($M_{(a)} = 0.034$, $SD_a = 0.020$), indicating MSPM has higher stability of DRR on portfolio(a). However, Fig 10 depicts that the right tail of MSPM's RstdDRR is fatter than that of ARL's RstdDRR, and the mean of MSPM's RstdDRR ($M_{(b)} = 0.049$, $SD_b = 0.027$) is larger than the mean of ARL's RstdDRR ($M_{(b)} = 0.032$, $SD_b = 0.022$). For more information, S1 and S2 Figs give the comparison between MSPM and ARL's RstdDRR for portfolio (a) and (b). As shown in S1 Fig, the RstdDRR of MSPM is less volatile than that of ARL, but it is the opposite case in S2 Fig.

Fig 9. For portfolio(a), histograms of MSPM and ARL's 5-day RstdDRR depict right-skewed distributions.
<https://doi.org/10.1371/journal.pone.0263689.g009>

Fig 10. For portfolio(b), histograms of MSPM and ARL's 5-day RstdDRR depict right-skewed distributions.
<https://doi.org/10.1371/journal.pone.0263689.g010>

Since the histograms in Figs 9 and 10 show skewed bell shapes, we use Shapiro-Wilk test [31] to confirm the normality of the distributions. After that, we use Levene's test [32] to examine the variance equality. We use Python's SciPy library to perform these two tests. By implementing Shapiro-Wilk test, we find that MSPM and ARL's RstdDRR are not statistically from normal distributions for both portfolios (p-values are less than 0.05). Moreover, according to Levene's test, MSPM and ARL's RstdDRR do not always have homogeneity of variance: for portfolio (a) they do, whereas for portfolio(b) they do not. With the assumptions verified, we perform the one-tail and two-sample Mann-Whitney U test [33] (a non-parametric version of unpaired t-test) to rigorously compare MSPM and ARL's stability of DRR, also using Python's SciPy library. For portfolio(a), because the mean RstdDRR of MSPM is less than the mean RstdDRR of ARL, the hypothesis H_0 is that MSPM has a lower or same stability than ARL (the group mean of RstdDRR of MSPM is greater or equal to that of ARL), and the alternative hypothesis H_a is that MSPM has higher stability than ARL (the group mean of RstdDRR of MSPM is less than that of ARL). For portfolio(b), because the mean RstdDRR of MSPM is higher than the mean RstdDRR of ARL, the hypothesis H_0 is that MSPM has higher or same stability than ARL (the group mean of RstdDRR of MSPM is less or equal to that of ARL), and the alternative hypothesis H_a is that MSPM has a lower stability than ARL (the group mean of RstdDRR of MSPM is greater than that of ARL). We set the significance level to be 0.05. If the p-value from the test is less than 0.05, we reject H_0 and accept H_a ; otherwise, we accept the null hypothesis H_0 . The detailed settings of the statistical test are:

- Statistical test: one-tail and two-sample Mann-Whitney U test
- For portfolio (a), null hypothesis
- For portfolio (a), alternative hypothesis
- For portfolio (b), null hypothesis
- For portfolio (b), alternative hypothesis
- Significance level: .05

As the results represented in Table 4, MSPM has significantly higher stability of DRR than ARL for portfolio(a) by rejecting H_0 and accepting H_a ($U_a = 25426.0$, $p - value = .005$). For portfolio(b), because H_0 is accepted ($U_b = 16209.0$, $p - value < .001$), we confirm that MSPM has lower stability of DRR than ARL. The conclusions are aligned with the MD in Table 3 and the underwater plots in S3–S6 Figs. which illustrate the drawdowns during year 2020. It is clear in S3 and S4 Figs that ARL has more frequent and intensive drawdowns for portfolio(a) compared to MSPM, but MSPM becomes the more volatile one for portfolio(b) according to S5 and S6 Figs. The results indicate that although MSPM achieves an outstanding performance in gaining capital returns, it does not naturally come with higher stability. However, low stability (or high risk) does not necessarily refer to danger. Since for both portfolio (a) and (b), MSPM has the highest Sortino ratios, which consider only the downside risk, MSPM's lower stability for portfolio (b) may come from a higher upside risk. In conclusion, there should be a trade-off between performance and stability, and this can be further investigated and considered in future studies.

Table 4. Results of statistical test on the RstdDRR of MSPM and ARL.
<https://doi.org/10.1371/journal.pone.0263689.t004>

EAM: Case study

To better understand how EAM contributes to SAM, we illustrate the position-holding information using the signals generated by the EAMs of portfolio (a) and (b) in Figs 11–15. The figures represent the five underlying assets: AAPL, AMD, GOOGL, NVDA, TSLA. In each plot, signals of Buying and Skipping are marked with cyan and orange circles, and the positions opened or closed are marked with either star or square symbols. The grey line is the normalized price movement. A position is opened when the first Buying signal is generated after the latest position has been closed. A position is closed when the first Closing signal is generated after a position has been opened and not yet been closed. We use dashed lines to divide different position-holding periods. If a position is profit-making based on the opening and closing prices, we color the period as light green (winning position), otherwise light red. Period of no-position will be left as blank. According to the results illustrated in the figures, the positions are opened and closed at just the right timings by the corresponding EAMs for most assets.

Fig 11. Signals and position-holding of AAPL's EAM.
<https://doi.org/10.1371/journal.pone.0263689.g011>

Fig 12. Signals and position-holding of AMD's EAM.
<https://doi.org/10.1371/journal.pone.0263689.g012>

Fig 13. Signals and position-holding of GOOGL's EAM.
<https://doi.org/10.1371/journal.pone.0263689.g013>

Fig 14. Signals and position-holding of NVDA's EAM.
<https://doi.org/10.1371/journal.pone.0263689.g014>

Fig 15. Signals and position-holding of TSLA's EAM.
<https://doi.org/10.1371/journal.pone.0263689.g015>

As shown in [Table 5](#), the number of positions opened by any EAM is less than ten, and the highest is NVDA and TSLA's eight opened positions. The most profit-making EAM is TSLA, with ARR of 799%. These results exemplify the high quality and reliability of the signals generated by the EAMs. The winning rates of all the five EAMs are more than 50%. Since averaged winning rate is 80%, it indicates that even with a mediocre averaged winning rate, SAM still can efficiently utilize the information generated by the EAMs and has the outperformance compared to ARL. The results also indicate that the MSPM can perform even better if we improve the winning rate of EAMs.

Table 5. Statistics of EAMs' position-holding during year 2020.
<https://doi.org/10.1371/journal.pone.0263689.t005>

Validation of EAM

As EAMs provide the trading signal-comprised information to SAMs, we intend to verify the indispensability of EAM by comparing the performance of MSPMs with and without EAMs. For this purpose, we set four different portfolios: (a), (b), (c), and (d), in which (c) and (d) are newly introduced. Portfolio(c) consists of three stocks: Alphabet, Nvidia, and Amazon (symbol codes: [GOOGL, NVDA, AMZN]), and portfolio(d) consists of three other stocks: Nvidia, Facebook, and Microsoft (symbol codes: [NVDA, FB, MSFT]). Two MSPMs/SAMs share the same EAM for the common stocks, which are NVDA and AMZN. The initial portfolio values are still set to be 10,000. [Fig 16](#) shows EAM-enabled and EAM-disabled MSPMs' the accumulated returns of different portfolios. As shown in the figure, EAM-enabled MSPMs always perform better than EAM-disabled MSPMs, and this conclusion can be reconfirmed by [Table 6](#). As listed in the table, EAM-enabled MSPMs largely outperform EAM-disabled MSPMs in terms of DRR, ARR, and SR. In terms of portfolio (d), EAM-enabled MSPM achieves ARR and SR of 115.6% and 2.45, whereas EAM-disabled MSPM's ARR and SR is -5.9% and 0.01. The results validate that the SAMs can only have an ideal performance with the trading signal-comprised information from EAMs.

Fig 16. Accumulated portfolio values of MSPMs, with and without EAMs, from back-testing for portfolio (a), (b), (c) and (d).
 For all the four portfolios, EAM-enabled MSPMs perform significantly better than EAM-disabled MSPMs.
<https://doi.org/10.1371/journal.pone.0263689.g016>

	Portfolio (a)		Portfolio (b)		Portfolio (c)		Portfolio (d)	
Metric	MSPM	MSPM(w/o)	MSPM	MSPM(w/o)	MSPM	MSPM(w/o)	MSPM	MSPM(w/o)
DRR (%)	6.480	5.850	6.550	5.920	6.480	5.850	6.340	5.690
ARR (%)	150.3	13.5	166.6	17.4	159.4	1.1	110.6	11.9
SR (%)	15.1	36.8	49.4	47.7	17.4	36.7	17.4	35.3
SR	2.86	0.01	4.18	1.90	2.97	0.10	2.45	0.01

MSPM: EAM enabled MSPM; MSPM(w/o): EAM disabled MSPM

<https://doi.org/10.1371/journal.pone.0263689.t006>

Table 6. Comparison of back-testing performance of EAM-enabled and EAM-disabled MSPMs.

<https://doi.org/10.1371/journal.pone.0263689.t006>

Discussion on scalability and reusability of MSPM

To address the issue of inefficient model training in RL-based PM, EAMs are designed to be independent and reusable. Once an EAM has been trained, it can be added to any SAM without retraining. For example, in the previous sections, portfolio(a) and portfolio(b) share one EAM in common: GOOGL, and it saves time and resources from redundant model training. On the other hand, to address the issues of ad-hoc and fixed model training in RL-based PM, MSPM allows the number of EAMs connected to any single SAM to be scaled up. In the EAM: Case study section, each EAM represents a single asset, and since these EAMs are trained, they are ready to be connected to any SAM. For example, to build a portfolio containing two assets, e.g., AAPL and TSLA, we can connect the corresponding two EAMs to an SAM to train and build the portfolio. Meanwhile, the rest of the EAMs can also be used in other portfolios. If later we want to scale up the volume of this portfolio to four assets, we simply add two more EAMs, e.g., GOOGL and NVDA, to the SAM without wasting time for training the EAMs again. Although SAM needs to be retrained once its volume is scaled up, the benefits brought by the EAMs are considerable since it has been validated in the previous section that the performance of an EAM-enabled SAM is largely improved compared to an EAM-disabled EAMs. Moreover, MSPM's scalability allows EAMs to accommodate the need for heterogeneous and alternative data input, like the sentiments data utilized in our research. Therefore, with MSPM's scalability and reusability to create dynamic and adaptive portfolios, researchers and portfolio managers can simultaneously perform capital reallocation for various portfolios of a large volume of assets at scale by parallel computing.

Limitations and future work

In this paper, to accommodate MSPM in sequential decision-making problems of PM, we only implement DQN and PPO to formalize the agents in EAM and SAM modules. We left the implementation of other algorithms in MSPM to future studies. Additionally, the trade-off between the stability of DRR and the performance metrics (ARR, DRR, or SR) may be further considered when designing the reward functions in future studies. We only implement the historical prices and sentiments data in this research, and we plan to utilize more heterogeneous data, e.g., satellite images, in the future studies.

Conclusion

We propose MSPM, a modularized multi-agent RL-based system, to bring scalability and reusability to financial portfolio management. We design and develop two types of modules in MSPM: EAM and SAM. EAM is an asset-dedicated module that takes heterogeneous data and utilizes a DQN-based agent to generate signal-comprised information. On the other hand, SAM is a decision-making module that receives stacked information from the connected EAMs to reallocate the assets in a portfolio. As EAMs can be combined and connected to any SAMs at will, with this modularized and reusable design, MSPM addresses the issue of ad-hoc, fixed, and inefficient model training in the existing RL-based methods. By experimenting, we confirm that MSPM outperforms various baselines in terms of the accumulated rate of return, daily rate of return, and Sortino ratio. Additionally, to exemplify the high quality and reliability of the signals generated by EAM, we inspect the position-holding of five different EAMs. Furthermore, we validate the necessity of EAM by back-testing and comparing the EAM-enabled and disabled MSPMs on four different portfolios. The experimental results prove that MSPM is qualified as a stepping stone to inspire more creative system designs in reinforcement learning-based financial portfolio management.

Supporting information

S1 Appendix. Model selection for EAM and hyperparameter tuning.

<https://doi.org/10.1371/journal.pone.0263689.s001>
(PDF)

S1 Fig. 5-day RstdDRR of Portfolio(a): MSPM versus ARL.

<https://doi.org/10.1371/journal.pone.0263689.s002>
(TIF)

S2 Fig. 5-day RstdDRR of Portfolio(b): MSPM versus ARL.

<https://doi.org/10.1371/journal.pone.0263689.s003>
(TIF)

S3 Fig. Underwater plot of MSPM for Portfolio(a).

<https://doi.org/10.1371/journal.pone.0263689.s004>
(TIF)

S4 Fig. Underwater plot of ARL for Portfolio(a).

<https://doi.org/10.1371/journal.pone.0263689.s005>
(TIF)

S5 Fig. Underwater plot of MSPM for Portfolio(b).

<https://doi.org/10.1371/journal.pone.0263689.s006>
(TIF)

S6 Fig. Underwater plot of ARL for Portfolio(b).

<https://doi.org/10.1371/journal.pone.0263689.s007>
(TIF)

Acknowledgments

This paper can also be accessed as a preprint on arXiv.org, with e-print number: 2102.03502 and arXiv.org—Non-exclusive license.

References

- Markowitz H. Portfolio Selection. *The Journal of Finance*. 1952;7(1):77–91.
[View Article](#) • [Google Scholar](#)
- Jiang Z, Xu D, Liang J. A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem; 2017. Available from: <https://arxiv.org/abs/1706.10059>.
- Silver D, Lever G, Heess N, Degris T, Wierstra D, Riedmiller M. Deterministic Policy Gradient Algorithms. In: *Proceedings of the 31st International Conference on International Conference on Machine Learning—Volume 32. ICML'14. JMLR.org*; 2014. p. I–387–I–395.
- Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, et al. Continuous control with deep reinforcement learning; 2019. Available from: <https://arxiv.org/abs/1509.02971>.
- Liang Z, Chen H, Zhu J, Jiang K, Li Y. Adversarial Deep Reinforcement Learning in Portfolio Management; 2018. Available from: <https://arxiv.org/abs/1808.09940>.
- Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal Policy Optimization Algorithms; 2017. Available from: <https://arxiv.org/abs/1707.06347>.
- Ye Y, Pei H, Wang B, Chen PY, Zhu Y, Xiao J, et al. Reinforcement-Learning Based Portfolio Management with Augmented Asset Movement Prediction States. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020;34(01):1112–1119.
[View Article](#) • [Google Scholar](#)
- López VF, Alonso N, Alonso L, Moreno MN. A Multiagent System for Efficient Portfolio Management. In: Demazeau Y, Dignum F, Corchado JM, Bajo J, Corchuelo R, Corchado E, et al., editors. *Trends in Practical Applications of Agents and Multiagent Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2010. p. 53–60.
- Sycara K, Decker K, Zeng D. Designing a Multi-Agent Portfolio Management System. In: *Proceedings of the AAAI Workshop on Internet Information Systems*; 1995.
- Lee J, Kim R, Yi SW, Kang J. MAPS: Multi-Agent reinforcement learning-based Portfolio management System. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. 2020. <https://doi.org/10.24963/ijcai.2020/623>
- Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, et al. Playing Atari with Deep Reinforcement Learning; 2013. Available from: <https://arxiv.org/abs/1312.5602>.
- Rokach L. Ensemble-based classifiers. *Artif Intell Rev*. 2010;33:1–39.
[View Article](#) • [Google Scholar](#)
- Liu Y, Liu Q, Zhao H, Pan Z, Liu C. Adaptive Quantitative Trading: An Imitative Deep Reinforcement Learning Approach. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020;34(02):2128–2135.
[View Article](#) • [Google Scholar](#)
- DeMiguel V, Garlappi L, Uppal R. Optimal Versus Naive Diversification: How Inefficient is the 1/N Portfolio Strategy? *The Review of Financial Studies*. 2007;22(5):1915–1953.
[View Article](#) • [Google Scholar](#)
- Fama EF. EFFICIENT CAPITAL MARKETS: A REVIEW OF THEORY AND EMPIRICAL WORK *. *Journal of Finance*. 1970;25:383–417.
[View Article](#) • [Google Scholar](#)
- Murphy JJ. *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. New York Institute of Finance; 1999.
- QuoteMedia. End-Of-Day Data; 2020. <https://data.nasdaq.com/data/EOD-end-of-day-us-stock-prices>.
- InfoTrie. FinSentS Web News Sentiment; 2021. <https://data.nasdaq.com/databases/NS1/data>.
- Araci D. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models; 2019. Available from: <https://arxiv.org/abs/1908.10063>.
- Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics; 2019. p. 4171–4186.
- Lapan M. *Deep Reinforcement Learning Hands-On: Apply Modern RL Methods, with Deep Q-Networks, Value Iteration, Policy Gradients, TRPO, AlphaGo Zero and More*. Packt Publishing; 2018.
- Hong S, Wu M, Zhou Y, Wang Q, Shang J, Li H, et al. ENCASE: An ENsemble CIASsifiEr for ECG classification using expert features and deep neural networks. In: *2017 Computing in Cardiology (CinC)*; 2017. p. 1–4.
- Wang Z, Schaul T, Hessel M, van Hasselt H, Lanctot M, de Freitas N. Dueling Network Architectures for Deep Reinforcement Learning; 2016. Available from: <https://arxiv.org/abs/1511.06581>.
- Hasselt Hv, Guez A, Silver D. Deep Reinforcement Learning with Double Q-Learning. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence. AAAI'16. AAAI Press*; 2016. p. 2094–2100.

25. Nair V, Hinton GE. Rectified Linear Units Improve Restricted Boltzmann Machines. In: Proceedings of the 27th International Conference on International Conference on Machine Learning. ICML'10. Madison, WI, USA: Omnipress; 2010. p. 807–814.
26. Ormos M, Urbán A. Performance analysis of log-optimal portfolio strategies with transaction costs. *Quantitative Finance*. 2013;13(10):1587–1597.
[View Article](#) • [Google Scholar](#)
27. Sortino FA, Price LN. Performance Measurement in a Downside Risk Framework. *The Journal of Investing*. 1994;3(3):59–64.
[View Article](#) • [Google Scholar](#)
28. Li B, Hoi SCH. Online Portfolio Selection: A Survey. *ACM Comput Surv*. 2014;46(3).
[View Article](#) • [Google Scholar](#)
29. Hudson and Thames Quantitative Research. Machine Learning Financial Laboratory (MIFinLab); 2021. <https://github.com/hudson-and-thames/mlfinlab>.
30. Law J, Smullen J. A Dictionary of Finance and Banking. Oxford University Press; 2008. Available from:
<https://www.oxfordreference.com/view/10.1093/acref/9780199229741.001.0001/acref-9780199229741>.
31. SHAPIRO SS, WILK MB. An analysis of variance test for normality (complete samples)†. *Biometrika*. 1965;52(3–4):591–611.
[View Article](#) • [Google Scholar](#)
32. Olkin I, Hotelling H. Contributions to probability and statistics: Essays in honor of Harold Hotelling. Stanford University Press; 1960.
33. Mann HB, Whitney DR. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*. 1947;18(1):50–60.
[View Article](#) • [Google Scholar](#)